

“Design and Development of a Digital Ration Card & PDS Tracker for Transparent Management of the Public Distribution System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

SURA RAHUL REDDY :2520030305

Y. YASHWANTH :2520030506

Under the Esteemed Guidance of

Dr. Purushottam

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Design and Development of a Digital Ration Card & PDS Tracker for Transparent Management of the Public Distribution System”** is a record of Bonafide work of **SURA RAHUL REDDY - 2520030305, Y. YASHWANTH - 2520030506** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

SURA RAHUL REDDY – 2520030305

Y. YASHWANTH – 2520030506

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled **“Design and Development of a Digital Ration Card & PDS Tracker for Transparent Management of the Public Distribution System”** is a bonafide work done and submitted by **SURA RAHUL REDDY - 2520030305, Y. YASHWANTH - 2520030506** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our Course Coordinator and our guide **Dr. Purushottam**, Department of Computer Science, which cannot be expressed in words for his tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Rao (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the Department for **Computer Science Engineering/ Computer Science and Information Technology** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted us directly or indirectly for successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

SURA RAHUL REDDY – 2520030305

Y. YASHWANTH – 2520030506

Abstract

The Public Distribution System (PDS) is a vital food-security programme, but its manual administration — physical ration cards, handwritten ledgers and periodic stock audits — leads to ghost ration cards, a lack of real-time stock tracking, poor transparency for beneficiaries and slow grievance redressal. These weaknesses allow food grains to be diverted and leave genuine families uncertain about their entitlements. To tackle this, our project focuses on the Design and Development of a Digital Ration Card & PDS Tracker. Our core aim was to build a web platform that modernises the PDS for beneficiaries, shop owners and administrators.

The platform provides three role-based dashboards – Admin, Shop Owner and Beneficiary – backed by REST APIs and a relational database. It digitises ration cards with a UNIQUE card number, manages monthly entitlements, tracks the real-time stock of every Fair Price Shop, records purchases that synchronise with stock, maintains a transparent transaction history, and offers a grievance module with Pending and Resolved tracking. Full CRUD operations are supported for users, shops, stock and grievances, with backend input validation and indexed SQL queries.

The system is built with HTML5, CSS3 and Vanilla JavaScript on the front end, Node.js and Express.js on the back end, and MySQL as the database, supported by Git & GitHub, Postman and VS Code. In summary, this project delivers a secure, transparent and user-friendly portal that replaces paper records, gives administrators real-time stock visibility to help prevent fraud, and lets beneficiaries view their entitlements. SMS/e-mail alerts and biometric authentication are proposed as future enhancements.

Index

S. No.	Chapters	Topics	Page.no
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience 1.7 Feasibility study	5-13
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	14-20
3	Technology Stack	3.1 Front-end (HTML/CSS/JavaScript) 3.2 Back-end (Node.js, Express.js) 3.3 Database (MySQL) 3.4 Version Control (Git, GitHub) 3.5 APIs or External services	21-28
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture	29-36
5	Design	5.1 Existing vs proposed system 5.2 ER Diagram / Database schema 5.3 Table structure	37-44
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	45-56
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	57-65
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	66-75
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	76-82
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	83-88
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	89-96
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	97-100
13	References	- Books, tutorials, APIs, documentation sites used	101

S. No.	Chapters	Topics	Page.no
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide - Geo Tag photos with guide - Review forms with guide signatures	102-103

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

The Public Distribution System (PDS) is one of the largest food-security programmes in India. Operating under the National Food Security Act, 2013, it supplies subsidised foodgrains such as rice, wheat and sugar to eligible households through a network of Fair Price Shops (FPS). The ration card is the document that links a household to this entitlement: it records the family, its category and the quantity of grain it may collect every month.

Although the scheme has a very wide reach, a large part of its day-to-day administration still depends on manual ledgers, physical ration cards and periodic paper-based stock audits. Beneficiaries often do not know how much grain they are entitled to in a given month, shop owners maintain stock registers by hand, and administrators can only see the real position of a shop long after the transactions have taken place. This gap between what is recorded on paper and what actually happens at the shop counter is where irregularities enter the system.

The Digital Ration Card & PDS Tracker was conceived to close this gap. It is a web platform that digitises ration cards, calculates and stores monthly entitlements, and tracks the stock of every Fair Price Shop in real time. Three role-based dashboards — for the Admin, the Shop Owner and the Beneficiary — give each stakeholder exactly the information and controls they need. The system is built with a widely supported, open-source stack: HTML5, CSS3 and Vanilla JavaScript on the front end, Node.js with Express.js on the back end, and MySQL as the relational database.

The project is developed as part of the course Database Systems Engineering and Distributed Backend Development (25CS1302E), and therefore places particular emphasis on sound relational database design — normalised tables, primary and foreign keys, UNIQUE constraints and indexed queries — together with a clean REST API layer that connects the database to the user interface.

1.2 Problem Statement

The manual Public Distribution System suffers from several interlinked problems that reduce its effectiveness and erode public trust:

- Ghost and duplicate ration cards: paper records make it difficult to detect cards that are duplicated, inactive or belong to non-existent households, so grain is issued against cards that should not exist.
- No real-time stock tracking: the stock held by a Fair Price Shop is verified only through periodic manual audits, which are error-prone and allow grain to be diverted to the open market between audits.
- Poor transparency: beneficiaries are generally unaware of their exact monthly quota and have no easy way to verify what has been recorded against their card.
- Slow and untraceable grievance redressal: complaints are submitted on paper or verbally, are difficult to track, and frequently remain unresolved for long periods.

The problem, therefore, is to design a digital platform that maintains a single trustworthy record of ration cards, entitlements, shop stock and transactions, and that makes this record visible to the people who depend on it.

1.3 Scope of the Project

The scope of the Digital Ration Card & PDS Tracker covers the following areas:

- Three web dashboards: an Admin dashboard, a Shop Owner dashboard and a Beneficiary dashboard, each with role-specific access.
- Digitisation of ration cards with a unique card number, household details and card status.
- Calculation and storage of monthly entitlements for each ration card and item.
- Real-time inventory management for every Fair Price Shop, updated automatically with each recorded purchase.
- A complete transaction history that beneficiaries and administrators can view.
- A grievance module with Pending and Resolved status tracking.
- Full CRUD (Create, Read, Update, Delete) operations on users, shops, stock and grievances.

The following items are outside the scope of the present work: dedicated Point-of-Sale (POS) hardware at the shop counter, biometric devices, and integration with any live government database. These are discussed as possible enhancements in Chapter 11.

1.4 Objectives of the Project

- Objective 1 — Digitise ration cards and track monthly entitlements accurately for every household.
- Objective 2 — Provide real-time Fair Price Shop inventory so that stock diversion can be detected and prevented.
- Objective 3 — Offer transparent transaction history and an effective grievance redressal mechanism for beneficiaries.
- Objective 4 — Enforce data integrity at the database level through strict schemas, UNIQUE constraints, foreign keys and backend input validation.

1.5 Importance of the Application

The importance of the application lies in the scale and sensitivity of the scheme it supports. Every kilogram of grain that is diverted deprives an eligible family of food security, and every undetected duplicate card increases the burden on public funds. By recording each entitlement and each purchase in a relational database, the system makes irregularities visible at the moment they occur rather than months later.

For beneficiaries, the application removes uncertainty: they can see their monthly quota, check what has been collected and raise a complaint from the same portal. For shop owners, it replaces handwritten registers with automatic stock updates and reduces reconciliation effort. For administrators, it provides a consolidated, real-time view across all shops, which supports faster audits and more informed decisions. Because the entire solution uses open-source technology, it can be adopted at very low cost.

1.6 Target Users / Audience

User Role	Description	Main Capabilities
Admin	Officials responsible for the PDS in an area.	Manage users, shops, ration cards and entitlements; monitor stock and transactions across all shops; resolve grievances.
Shop Owner	Operator of a Fair Price Shop.	View and update shop stock; record purchases against ration cards; view transaction logs of the shop.
Beneficiary	Ration card holder (household).	View card details and monthly entitlement; see purchase history; raise and track grievances.

Since many beneficiaries are not technically experienced, the front end is designed to be simple, uncluttered and easy to navigate.

1.7 Feasibility Study

- **Technical feasibility:** Highly feasible. The project uses an established and widely supported stack (HTML/CSS/JavaScript, Node.js, MySQL) with abundant documentation and community support.
- **Economic feasibility:** Very cost-effective. All tools are open source, free-tier hosting is sufficient for a prototype, and there are no licensing costs.
- **Operational feasibility:** The front end is user-friendly for less tech-savvy users, while the back end offers streamlined management for administrators.
- **Schedule feasibility:** The scope is modular — APIs, database design and front-end integration can be developed and tested independently and completed within the semester.

CHAPTER -1 INTRODUCTION (CONTINUED)

1.8 Extended Project Overview

The Digital Ration Card & PDS Tracker can be understood as a connected information system rather than a set of independent web pages. The central flow begins with a household and its ration card, continues through monthly entitlement and Fair Price Shop stock, and ends with a recorded transaction. A consistent digital record is maintained at each stage so that the information shown to the beneficiary, shop owner and administrator is based on the same underlying database.

1.8.1 Purpose of the Extended Design

The additional design discussion expands the original project explanation without changing the implemented technology choices. It describes how the existing modules relate to one another, why the selected database constraints are useful, and how the documented workflows can be examined during review. The emphasis remains on the prototype already described in the report.

1.8.2 Information Continuity

Information continuity is important because one action can influence several views. For example, when a purchase is accepted, the transaction history receives a record, the collected quantity associated with the entitlement changes, and the shop stock decreases. The beneficiary can therefore see the updated balance while the shop owner and administrator see the corresponding stock movement.

1.8.3 Stakeholder Perspective

The three roles have deliberately different information needs. The Admin requires a broad operational view, the Shop Owner needs a focused interface for stock and purchases, and the Beneficiary needs a simple view of household information, entitlement, history and grievances. This separation supports clarity while also reinforcing the role-based access rules implemented by the back end.

1.8.4 Engineering Perspective

From an engineering perspective, the project combines relational modelling, REST communication, validation, authentication, authorisation, transaction processing and API testing. The value of the application lies in the way these concepts operate together: the browser collects input, Express applies application rules, and MySQL provides persistent structured records.

CHAPTER -1 INTRODUCTION (CONTINUED)

1.8 Project Context

The project can be viewed as a digital record-management system built around a simple but important chain of information: a household owns a ration card, the card has an entitlement, the household receives items through a shop, the shop has stock, and each issue of an item becomes a transaction. Keeping this chain consistent is the central engineering problem.

1.9 Stakeholder View

The Admin requires broad visibility and controlled management functions. The Shop Owner needs operational screens for stock and purchases. The Beneficiary needs a focused view of personal card information, monthly entitlement, history and grievances. The role model therefore follows the responsibilities already defined in the original implementation rather than creating a common dashboard with excessive controls.

1.10 Information Flow

A typical transaction begins with a card identifier and an item quantity. The server validates the request, verifies entitlement and available shop stock, records the transaction and updates the relevant quantities. This sequence makes the database the authoritative source while the browser acts as a presentation and input layer.

1.11 Engineering Perspective

The project demonstrates several database and backend concepts together: relational modelling, constraints, indexes, CRUD operations, REST communication, authentication, authorisation, transaction handling and API testing. Its educational value comes from combining these concepts into a single coherent application.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -1 INTRODUCTION (CONTINUED)

1.12 Functional Requirements

Functional requirements describe actions the system must support. The existing report identifies user management, ration-card registration, entitlement handling, shop and stock management, transaction recording, transaction history and grievance handling. These functions can be decomposed into smaller operations so that each can be implemented, tested and maintained independently.

1.13 Non-Functional Requirements

The prototype also has non-functional expectations. It should respond predictably, protect role-specific information, preserve data integrity, remain understandable to less technical users and operate with modest hardware. The use of HTML, CSS, JavaScript, Node.js, Express.js and MySQL keeps the architecture lightweight.

1.14 Data Integrity Requirements

Data integrity is especially important because the same purchase affects several pieces of information. A transaction must refer to valid entities, quantities must be positive, a card number must be unique, and a purchase should not be committed if the required stock or entitlement is unavailable. Database constraints and application validation therefore complement one another.

1.15 Requirement Traceability

A useful way to evaluate the implementation is to map each requirement to a module and a test. For example, unique card registration maps to the ration-card module and duplicate-card test; stock synchronisation maps to the transaction module and stock-reduction test; grievance tracking maps to the grievance module and status-change test.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -1 INTRODUCTION (CONTINUED)

1.16 Admin Role

The Admin is the management role. The report assigns this role responsibility for users, shops, ration cards, entitlements, stock visibility, transactions and grievance resolution. The important design principle is broad visibility without allowing the beneficiary or shop owner to inherit administrative controls.

1.17 Shop Owner Role

The Shop Owner represents the operational Fair Price Shop interface. The role needs access to the stock belonging to the owner's shop and the ability to record purchases. The server must therefore check not only that the user is a shop owner, but also that the requested shop belongs to that user before allowing sensitive updates.

1.18 Beneficiary Role

The Beneficiary is deliberately given a narrower interface. The user can view the card, entitlement and purchase history belonging to the household and submit or track grievances. Restricting the scope reduces accidental exposure of other households and keeps the interface manageable.

1.19 Authorisation Matrix

Role-based access is implemented as a server-side rule rather than a visual convention. Hiding a button in the browser is not sufficient protection. The API must reject unauthorised calls even if a user manually constructs a request, which is why Express middleware is used before protected route handlers.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -1 INTRODUCTION (CONTINUED)

1.20 Beneficiary Journey

A beneficiary logs in, reaches the beneficiary dashboard, reviews the digital ration card, checks the current month's quota and collected quantity, and reviews previous transactions. If an issue occurs, the beneficiary can submit a grievance and later observe whether it remains Pending or has been marked Resolved.

1.21 Shop Owner Journey

The Shop Owner logs in, checks current stock, receives a purchase request and enters the card, item and quantity. The server performs the entitlement and stock checks before committing the purchase. The resulting transaction becomes part of the shop history and the stock quantity changes immediately.

1.22 Admin Journey

The Admin uses the broader dashboard to register or manage users and shops, register cards, add or correct stock, inspect transactions and resolve grievances. Because these operations affect system-wide records, the Admin role is protected by the same authentication and authorisation layer.

1.23 End-to-End Workflow

The three journeys are connected by shared database records. A card registered by the Admin becomes visible to the beneficiary; stock entered for a shop becomes available to the Shop Owner; and a completed transaction can be viewed in the appropriate histories. This shared record is what makes the application a system rather than a collection of disconnected screens.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -1 INTRODUCTION (CONTINUED)

1.24 Choice of Vanilla JavaScript

The report intentionally uses HTML5, CSS3 and Vanilla JavaScript rather than a heavy front-end framework. For a compact academic application, this reduces build complexity and makes the relationship between DOM elements, browser events and API calls easy to inspect.

1.25 Layout Principles

The dashboards should prioritise the most important information first: identity, role, current entitlement or stock status, recent activity and available actions. Secondary information can remain below the main cards or inside tables. This hierarchy helps users complete common tasks without navigating through unnecessary screens.

1.26 Forms and Validation

Browser-side validation gives immediate feedback for missing or malformed values, while server-side validation remains authoritative. This two-layer approach improves usability without trusting client input. A form can therefore reject obvious mistakes quickly while the server independently checks all security and integrity rules.

1.27 Error and Success Feedback

A successful purchase, rejected quantity or submitted grievance should produce visible feedback close to the action that caused it. The original implementation keeps messages on the same page, which avoids unnecessary navigation and makes the result of each operation clear.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application is a lightweight web platform and does not need specialised hardware. The requirements below describe a comfortable development and demonstration environment, together with what an end user needs to access the system.

Component	Development Machine	End User Device
Processor	Intel Core i3 / AMD Ryzen 3 or higher	Any modern dual-core processor
RAM	8 GB recommended (4 GB minimum)	2 GB or more
Storage	At least 2 GB free for tools, packages and the database	Not significant
Display	1366 x 768 or higher	Desktop, laptop, tablet or smartphone
Network	Broadband connection for package installation and testing	Internet or local network access

No Point-of-Sale machine, fingerprint scanner or barcode reader is required in the present version.

2.2 Software Requirements

Category	Software
Operating System	Windows 10/11, Linux or macOS
Front-End	HTML5, CSS3, Vanilla JavaScript (ES6)
Back-End Runtime	Node.js (LTS version) with npm
Back-End Framework	Express.js (REST APIs)
Database	MySQL (relational database server) and MySQL Workbench
Version Control	Git and GitHub
API Testing	Postman
Code Editor	Visual Studio Code
Browser	Google Chrome, Microsoft Edge or Mozilla Firefox (latest versions)

2.3 Development Tools and Frameworks

Visual Studio Code was used as the primary editor, with extensions for JavaScript linting and formatting. Node.js and npm manage the server runtime and third-party packages, while Express.js provides routing and middleware for the REST APIs. MySQL Workbench was used to design the schema, run queries and inspect data. Git and GitHub provided version control and a shared repository for the two team members, and Postman was used to design, send and verify API requests independently of the user interface.

CHAPTER -2 SYSTEM REQUIREMENTS (CONTINUED)

2.4 Device Variety

The stated end-user environment includes desktops, laptops, tablets and smartphones. The interface should therefore avoid depending on large fixed-width controls. Tables should remain readable, forms should stack when the viewport becomes narrow, and the most important values should remain visible without excessive horizontal scrolling.

2.5 Low-End Hardware

The selected front-end stack has a small runtime footprint compared with a large client-side framework. Static HTML, CSS and JavaScript can be delivered efficiently, while the database and business rules remain on the server.

2.6 Slow Networks

The application should avoid repeatedly downloading unchanged assets and should request only the data needed by the current dashboard. Because the API returns JSON, the browser can update the relevant table or card without reloading the entire page.

2.7 Practical UI Guidance

Clear labels, moderate font sizes, sufficient spacing and visible status indicators are particularly useful for a public-service application. The goal is not decorative complexity; it is predictable interaction for users who may access the portal only occasionally.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -2 SYSTEM REQUIREMENTS (CONTINUED)

2.8 Learnability

A first-time beneficiary should be able to identify the card details, current entitlement and purchase history without reading technical documentation. Clear labels and a small number of primary actions support this goal.

2.9 Efficiency

Frequent actions should require as few screens as possible. For a Shop Owner, recording a purchase should be a direct flow from card number and item selection to confirmation.

2.10 Error Recovery

When an operation fails, the interface should preserve entered information where practical and explain the reason for rejection. This is more useful than a generic “something went wrong” message.

2.11 Accessibility Direction

A future refinement should check keyboard navigation, readable contrast, form labels and responsive behaviour. Multilingual support is already listed as a future enhancement in the original report.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -2 SYSTEM REQUIREMENTS (CONTINUED)

2.12 Readability

The interface should use clear typography and sufficient spacing, particularly because the application may be accessed by users with different levels of digital familiarity.

2.13 Colour Usage

Status colours should not be the only way to communicate information. Labels such as Pending and Resolved should accompany any visual distinction so the state remains understandable.

2.14 Keyboard Access

Forms, buttons and navigation controls should have a predictable focus order. This improves accessibility and also helps users who prefer keyboard interaction.

2.15 Language Support

The current prototype is described as single-language. A future localisation layer could move interface strings into resource files so that translations can be added without rewriting the application logic.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -2 SYSTEM REQUIREMENTS (CONTINUED)

2.16 Initial Planning

The project can be decomposed into database design, API design, authentication, front-end dashboards, transaction logic, grievance handling and testing. This follows the modular structure already described in the report.

2.17 Parallel Work

Team members can work on separate modules when the API contract and database schema are agreed early. This allows the front end to build against stable endpoint definitions while backend modules are implemented.

2.18 Integration Stage

Once individual modules work, integration focuses on shared flows such as card registration followed by purchase. This is where mismatches in field names, permissions or assumptions become visible.

2.19 Final Validation

The final stage should repeat the test matrix, inspect the database, verify the user journeys and check the installation procedure. The result is a demonstrable system rather than only a collection of code files.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -2 SYSTEM REQUIREMENTS (CONTINUED)

2.20 Technical Risk

Database inconsistency is a high-impact technical risk because a single purchase touches multiple records. The transaction mechanism reduces this risk.

2.21 Security Risk

Incorrect role enforcement could expose administrative operations. Centralised authorisation middleware and server-side checks reduce this risk.

2.22 Data Risk

Using unrealistic or inconsistent test data can hide relationship problems. A structured synthetic dataset improves test quality.

2.23 Deployment Risk

Missing environment variables or incompatible database configuration can prevent startup. The deployment chapter therefore separates configuration from source code and provides a setup sequence.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Risk	Potential Impact	Current Mitigation	Further Action
DB inconsistency	Incorrect stock/history	Transactions + rollback	Load/concurrency tests
Unauthorised access	Data exposure	Role middleware	Security audit
Bad test data	Hidden defects	Structured synthetic data	Seed scripts
Config error	Deployment failure	.env configuration	Deployment checklist

CHAPTER -2 SYSTEM REQUIREMENTS (CONTINUED)

2.24 Database Checklist

Verify primary keys, foreign keys, unique constraints, indexes, valid status values and transaction behaviour.

2.25 API Checklist

Verify authentication, authorisation, input validation, status codes, JSON responses, error handling and rollback behaviour.

2.26 Front-End Checklist

Verify login flow, dashboard visibility, form validation, feedback messages, responsive layout and role-specific navigation.

2.27 Documentation Checklist

Verify installation steps, environment variables, API descriptions, test cases, known limitations and future enhancements.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Area	Check
Database	Keys, constraints, indexes, relationships
Backend	Routes, validation, auth, transactions
Frontend	Forms, dashboards, feedback, responsiveness
Testing	Positive and negative cases
Documentation	Setup, API, limitations, future work

CHAPTER -3 TECHNOLOGY STACK

3.1 Front-End Design

The front end is built using HTML5, CSS3 and Vanilla JavaScript, without any heavy framework. HTML5 defines the structure of the pages, CSS3 provides the layout, colours and responsive behaviour, and JavaScript handles user interaction and communication with the back end through the Fetch API. This choice keeps the application light, easy to host and easy to understand, which is especially valuable for users on low-end devices and slow networks.

Separate dashboards are provided for the Admin, the Shop Owner and the Beneficiary. Each dashboard shows only the menus and data that are relevant to the role, which keeps the interface simple for non-technical users.

3.2 Back-End Design

The back end is developed with Node.js and Express.js. Node.js offers an event-driven, non-blocking runtime that handles many concurrent requests efficiently, and Express.js provides a minimal and flexible framework for defining routes, middleware and error handling. The server exposes REST APIs that accept and return JSON, validate every input, apply role-based access rules and communicate with MySQL through a connection pool.

The back-end code is organised by responsibility — routes, controllers, middleware and database configuration — so that each module can be developed and tested separately.

3.3 Database Design

MySQL is used as the relational database because the PDS data is naturally structured and highly interrelated: users own shops, households hold ration cards, cards have entitlements, and shops record transactions against cards. A relational model with primary keys, foreign keys and constraints guarantees the integrity of these relationships.

- UNIQUE constraint on the ration card number prevents duplicate or ghost cards from being registered.
- Foreign keys ensure that every transaction refers to a valid card, shop and item.
- Indexes on frequently searched columns such as card number, shop and month make lookups fast.
- ENUM columns restrict values such as user role and grievance status to valid options.

3.4 Version Control

Git was used for local version control and GitHub as the remote repository. Both team members worked on separate branches for features such as authentication, stock management and grievances, and merged them into the main branch after testing. Descriptive commit messages created a clear history of the project and made it easy to revert faulty changes.

3.5 APIs or External Services

The system is built around its own REST API layer. The API exposes resources such as authentication, ration cards, entitlements, shops, stock, transactions and grievances. Postman was used throughout development to

verify these APIs. The present version does not depend on any paid third-party service; SMS or e-mail gateways and Aadhaar-based authentication are proposed as future integrations.

Layer	Technology	Purpose
Front-End	HTML5, CSS3, Vanilla JavaScript	User interface and dashboards
Back-End	Node.js, Express.js	REST APIs, validation, business logic
Database	MySQL	Persistent relational storage
Tools	Git & GitHub, Postman, VS Code	Version control, API testing, development

CHAPTER -3 TECHNOLOGY STACK (CONTINUED)

3.6 Resource-Oriented Endpoints

The API is organised around resources such as users, cards, stock, transactions and grievances. HTTP methods express the intended operation: GET for retrieval, POST for creation, PUT for updates and DELETE for removal.

3.7 Consistent Responses

A predictable JSON response makes front-end integration easier. Success responses should contain the requested record or a concise message, while errors should contain a status code and an explanation that is useful during development and testing.

3.8 Validation Boundary

The API should never assume that a browser has validated its input correctly. Every important field must be checked on the server before it reaches SQL statements or business logic.

3.9 API Maintainability

Keeping routes separated by responsibility helps the team test and modify one module without changing unrelated endpoints. The same separation also makes it easier to document the API for a future developer.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -3 TECHNOLOGY STACK (CONTINUED)

3.10 Separation of Concerns

The existing project separates routes, controllers, middleware and database configuration by responsibility. This reduces the amount of unrelated code that must be changed for a single feature.

3.11 Naming

Consistent names for routes, variables, database columns and front-end elements reduce confusion. The existing API naming pattern provides a predictable resource-oriented structure.

3.12 Reusable Middleware

Authentication and role checks are naturally reusable. Implementing them once avoids copying security logic into every route.

3.13 Documentation

The expanded report itself becomes a maintenance aid by describing the data model, endpoints, workflows and known limitations. A future developer can use these sections as a starting point before changing code.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -3 TECHNOLOGY STACK (CONTINUED)

3.14 Branching

The report states that the two team members used separate branches for features such as authentication, stock management and grievances. Feature branches reduce the chance that unfinished work immediately affects the main branch.

3.15 Commits

Small, descriptive commits make it easier to understand why a change was introduced. They also provide recovery points if a new change introduces a regression.

3.16 Integration

Before merging, a feature should be tested through the relevant API and, where applicable, through the complete browser workflow. This connects version-control practice with software-quality practice.

3.17 Repository Hygiene

Environment files containing credentials should remain outside version control. Source code, schema scripts, documentation and safe configuration examples can be versioned instead.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -3 TECHNOLOGY STACK (CONTINUED)

3.18 Configuration Separation

The same code can run with different database credentials by keeping configuration in environment variables. This prevents environment-specific values from being embedded in source files.

3.19 Example Variables

The existing report lists PORT, DB_HOST, DB_USER, DB_PASSWORD, DB_NAME and JWT_SECRET. Each variable should have a clear purpose and should be documented without exposing the real secret.

3.20 Development Versus Production

Development may use localhost and a local database, while production uses a managed database host. Keeping these settings external means the application code does not need to change between environments.

3.21 Secret Rotation

A production system should support changing secrets without rewriting source code. This is another reason to keep secrets in deployment configuration rather than the repository.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

```
PORT=3000
DB_HOST=localhost
DB_USER=pds_user
DB_PASSWORD=<your_password>
DB_NAME=pds_tracker
JWT_SECRET=<long_random_string>
```

CHAPTER -3 TECHNOLOGY STACK (CONTINUED)

3.22 Collection Organisation

A Postman collection can be divided into folders for Authentication, Users, Cards, Shops, Stock, Transactions and Grievances. This mirrors the backend resource structure.

3.23 Environment Variables

A Postman environment can hold the base URL and token used during testing. This avoids editing every request when the server address changes.

3.24 Positive Requests

Each endpoint should have at least one valid request. The response status, response body and database state can be recorded as evidence.

3.25 Negative Requests

Invalid quantities, missing fields, duplicate identifiers and unauthorised roles should also be represented. Negative tests are important because many integrity problems appear only when input is incorrect.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Folder	Representative Requests
Authentication	Login
Users	Create / Read / Update / Delete
Cards	Create / Search / Entitlements
Stock	Read / Update
Transactions	Create / History
Grievances	Create / Resolve

CHAPTER -3 TECHNOLOGY STACK (CONTINUED)

3.26 Success Codes

The application can use 200-level responses for successful retrieval and updates and 201 for successful creation. The exact response should match the endpoint behaviour.

3.27 Client Errors

A 400 response is appropriate for malformed or invalid input. A 403 response is used in the report for role-based access denial.

3.28 Not Found

A missing card, shop or other resource should produce a controlled not-found response rather than a server exception. This keeps the API predictable.

3.29 Server Errors

Unexpected database or application exceptions should produce a controlled server error while detailed diagnostics remain in server-side logs.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Class	Meaning	Project Example
2xx	Successful request	Purchase recorded
400	Invalid input	Missing quantity
403	Forbidden	Beneficiary calls Admin route
404	Resource not found	Unknown card
5xx	Unexpected server error	Database failure

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

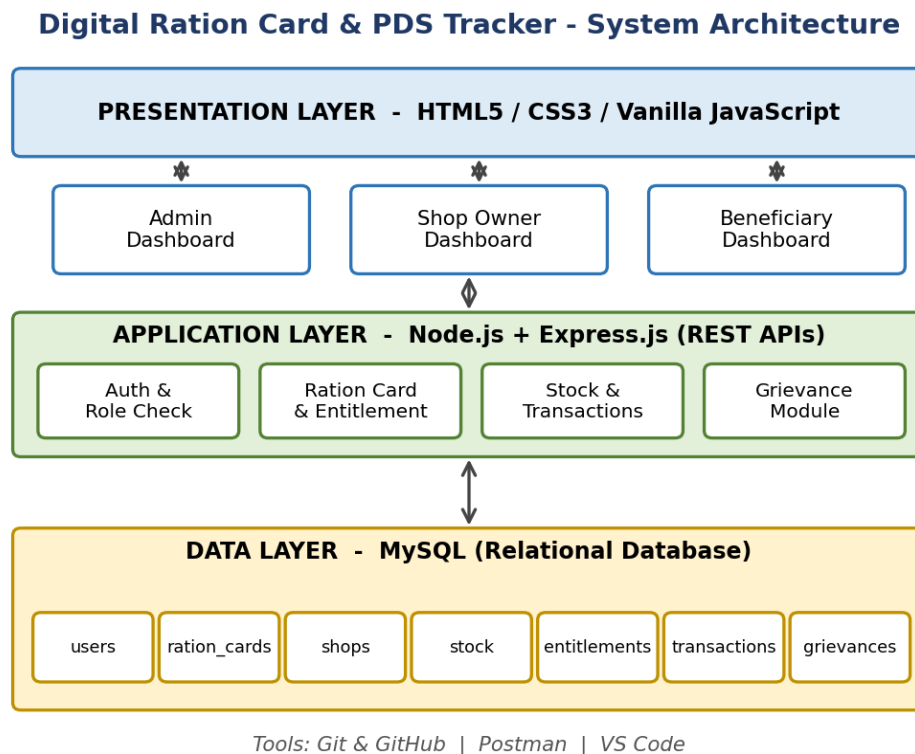


Figure 4.1: High-level architecture of the Digital Ration Card & PDS Tracker

4.2 Description of Each Layer (Frontend, Backend, Database)

The system follows a three-layer architecture that separates the user interface, the business logic and the data storage. This separation makes the application modular, easier to test and easier to extend.

Presentation layer. The presentation layer consists of HTML, CSS and Vanilla JavaScript pages. It contains the login page and the three role-based dashboards. This layer never talks to the database directly; it sends HTTP requests to the REST API and renders the JSON responses.

Application layer. The application layer runs on Node.js and Express.js. Middleware first authenticates the request and checks the role of the user, then validates the input, and finally the route handler executes the business logic — for example, checking an entitlement, recording a transaction and reducing shop stock. Errors are returned in a consistent JSON format with suitable HTTP status codes.

Data layer. The data layer is the MySQL database. It stores users, ration cards, shops, items, stock, entitlements, transactions and grievances in normalised tables. Constraints and indexes at this level guarantee that invalid data cannot be stored even if a validation check in the application is bypassed.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

During development, all three layers run on a single machine: the Express server on a local port, MySQL as a local service, and the front-end files served as static assets by Express. For a production-style deployment, the same structure maps onto a free-tier hosting platform for the Node.js server and a managed MySQL instance,

with the code stored on GitHub. Code pushed to the main branch can be redeployed automatically by the hosting platform, giving a simple continuous-deployment workflow.

CHAPTER -4 SYSTEM ARCHITECTURE (CONTINUED)

4.4 Request Entry

A browser action generates an HTTP request to an Express route. The request contains the required parameters or JSON body and, for protected resources, the authentication token. The server receives the request before any database modification takes place.

4.5 Middleware Processing

Authentication middleware verifies the identity represented by the token. Authorisation middleware or route-level checks then determine whether the role is permitted to access the operation. Validation follows so malformed or incomplete values are rejected early.

4.6 Business Logic

The route handler applies the project rules. A purchase route, for example, must identify the card, shop and item, verify the requested quantity, check entitlement and stock, create the transaction and update the related quantities.

4.7 Database Response

After successful database operations, the server returns a suitable status code and JSON message or data object. If an error occurs during a multi-step transaction, rollback is used so partial updates are not left behind.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -4 SYSTEM ARCHITECTURE (CONTINUED)

4.8 Entity Separation

The database separates users, ration cards, shops, items, stock, entitlements, transactions and grievances. This avoids storing repeated information in every row and gives each concept a clear home.

4.9 Keys and Relationships

Primary keys uniquely identify rows. Foreign keys connect related entities. For example, a transaction references a card, shop and item, while stock references a shop and item. These relationships allow the database to enforce consistency between modules.

4.10 Avoiding Repeated Master Data

Commodity names belong in the items table instead of being copied into every stock and transaction row. This prevents small spelling differences from creating apparently different items and makes it possible to change master information in one place.

4.11 Practical Benefits

The relational structure makes reporting and searching predictable. It also supports future growth because additional items, shops, cards and transactions can be added without changing the basic structure of existing records.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -4 SYSTEM ARCHITECTURE (CONTINUED)

4.12 Why Indexes Matter

The report explicitly indexes the ration card number because card lookup is a common operation. An index lets the database locate matching values without scanning every row in the table.

4.13 Candidate Search Fields

As the dataset grows, other frequently searched fields can also be considered for indexes, such as shop code, transaction date or combinations used by recurring dashboard queries. Indexes should be added based on actual query patterns rather than added indiscriminately.

4.14 Write Cost

Indexes improve reads but add some work to inserts and updates because index entries must also be maintained. For this project the expected transaction volume is moderate, so targeted indexes provide a reasonable balance.

4.15 Future Optimisation

If transaction history becomes very large, the report's proposed optimisations—partitioning, caching and query tuning—could be evaluated. These should be introduced only after measuring real query performance and identifying the actual bottleneck.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -4 SYSTEM ARCHITECTURE (CONTINUED)

4.16 Multi-Step Purchase Operation

A purchase is not a single database change. It can affect the transaction history, the entitlement's collected quantity and the shop's stock. Treating these changes as one logical operation prevents the database from representing an impossible partial state.

4.17 Commit and Rollback

The implementation starts a database transaction, performs the required checks and updates, and commits only after the operations succeed. If an exception occurs, the connection rolls back the work. This is the reason the original report emphasises the transaction module as the core of the system.

4.18 Example Failure

Suppose stock is available but the entitlement check fails. The transaction must not be inserted and the stock must not be reduced. Similarly, if an update fails after the transaction row is created, rollback prevents a history entry from surviving without the corresponding quantity changes.

4.19 Consistency as a Design Principle

The broader lesson is that business operations should be mapped to database transactions whenever several changes represent one user-visible event. This is a key database-engineering principle demonstrated by the project.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -4 SYSTEM ARCHITECTURE (CONTINUED)

4.20 Identifier Fields

Fields such as user_id, card_id, shop_id, item_id, entitlement_id, txn_id and grievance_id are surrogate identifiers used to connect records. They are primarily for database relationships and should not be confused with human-facing codes.

4.21 Human-Facing Identifiers

The ration card number and shop code are designed for lookup and communication with users. Their uniqueness makes them suitable for search operations.

4.22 Quantity Fields

quota_kg, collected_kg and quantity_kg represent different meanings. Quota is the allowance, collected is consumption against that allowance, and transaction or stock quantity represents movement in the relevant context.

4.23 Status Fields

Card status and grievance status represent lifecycle state. Keeping states explicit makes filtering and dashboard presentation straightforward.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Field	Meaning	Example Use
card_no	Unique ration card identifier	Search a beneficiary record
quota_kg	Monthly entitlement	Display allowance
collected_kg	Amount already collected	Calculate balance
quantity_kg	Quantity in stock/transaction	Update inventory
status	Lifecycle state	Filter active/pending records

CHAPTER -4 SYSTEM ARCHITECTURE (CONTINUED)

4.24 User to Card

A user may be the head of a ration card. The foreign key ensures that the referenced user exists.

4.25 Card to Entitlement

A card can have multiple entitlement records because each item and period is represented separately.

4.26 Shop to Stock

A shop has stock entries for the items it holds. The stock table connects shop and item.

4.27 Card to Transaction

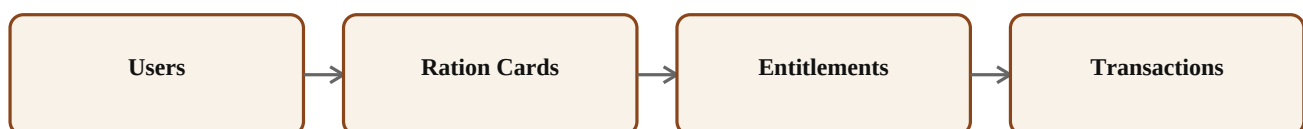
Transactions connect a beneficiary's card with the shop and item involved in the purchase, providing a historical record.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.



Illustrative workflow / component relationship

CHAPTER -5 DESIGN

5.1 Existing System vs Proposed System

Feature	Existing System	Proposed System
Record Keeping	Manual ledgers and physical ration cards	Digital records stored in a relational database
Inventory Tracking	Periodic manual audits (error-prone)	Real-time stock updates across all shops
Transparency	Entitlements unknown to beneficiaries	Monthly quotas and purchase history visible through the portal
Grievance Redressal	Slow, paper-based and hard to track	Online portal with Pending / Resolved tracking
Limitation / Gap	High scope for corruption and stock diversion	Role-based access and strict database schemas

5.2 ER Diagram / Database Schema

Entity-Relationship Diagram

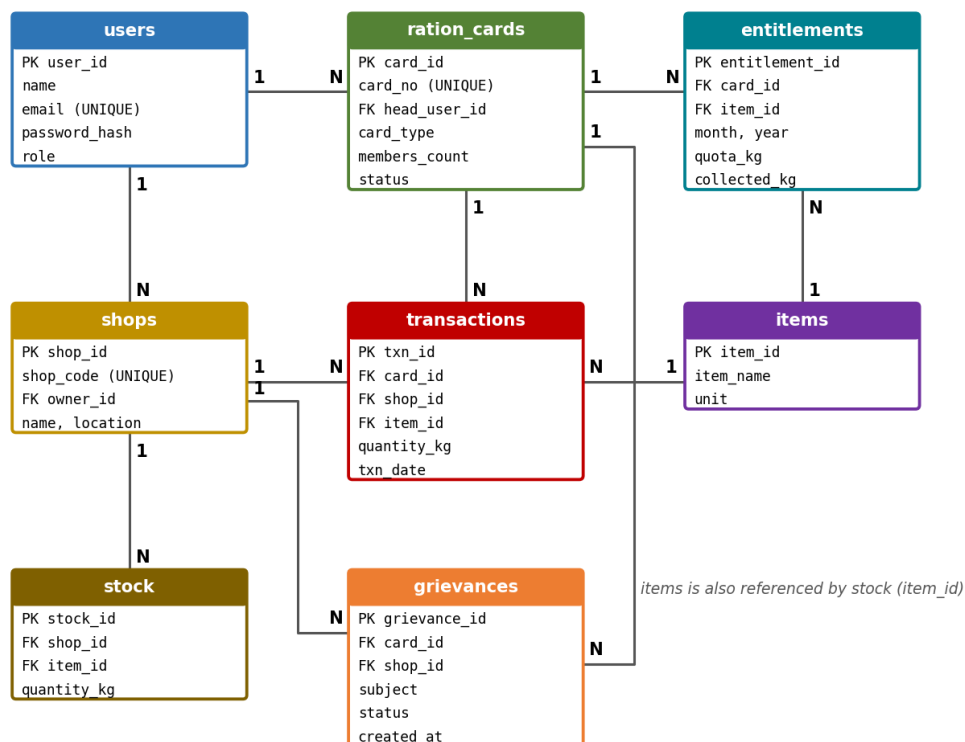


Figure 5.1: Entity-Relationship diagram of the database

The database contains eight tables. A user may own many shops and may be the head of many ration cards. A ration card has many monthly entitlements, many transactions and many grievances. A shop holds stock for many items and records many transactions. The items table is the master list of commodities such as rice, wheat and sugar.

5.3 Table Structure

Table	Key Columns	Notes
users	user_id (PK), name, email, password_hash, role	role is ENUM('admin','shop_owner','beneficiary'); email is UNIQUE
ration_cards	card_id (PK), card_no, head_user_id (FK), card_type, members_count, status	card_no is UNIQUE and indexed
shops	shop_id (PK), shop_code, owner_id (FK), name, location	shop_code is UNIQUE
items	item_id (PK), item_name, unit	Master list of commodities
stock	stock_id (PK), shop_id (FK), item_id (FK), quantity_kg	One row per shop-item pair
entitlements	entitlement_id (PK), card_id (FK), item_id (FK), month, year, quota_kg, collected_kg	Monthly quota and quantity already collected
transactions	txn_id (PK), card_id (FK), shop_id (FK), item_id (FK), quantity_kg, txn_date	Immutable purchase history
grievances	grievance_id (PK), card_id (FK), shop_id (FK), subject, description, status, created_at	status is ENUM('Pending','Resolved')

CHAPTER -5 DESIGN (CONTINUED)

5.4 Users Table

The users table is the identity foundation of the application. It stores a primary key, name, unique email, password hash and role. The role is restricted to the supported application roles.

5.5 Ration Cards Table

The ration_cards table connects a household to the user who heads the card and stores card type, member count and status. The card number is unique and indexed because it is a common lookup value.

5.6 Constraint Strategy

Constraints provide protection close to the data. A UNIQUE constraint prevents duplicate card numbers, a foreign key ensures the head user exists, and the positive member-count check prevents an obviously invalid household size.

5.7 Example DDL

The following representative schema follows the structure already included in the report. It is shown here as expanded documentation rather than a new database design.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

```
CREATE TABLE ration_cards (  
  card_id INT AUTO_INCREMENT PRIMARY KEY,  
  card_no VARCHAR(20) NOT NULL UNIQUE,  
  head_user_id INT NOT NULL,  
  card_type VARCHAR(20) NOT NULL,  
  members_count INT NOT NULL CHECK (members_count > 0),  
  status ENUM('Active','Inactive') DEFAULT 'Active',  
  FOREIGN KEY (head_user_id) REFERENCES users(user_id)  
);  
CREATE INDEX idx_card_no ON ration_cards(card_no);
```

CHAPTER -5 DESIGN (CONTINUED)

5.8 Shops

The shops table identifies each Fair Price Shop and links it to its owner. The unique shop code is useful for human-facing identification and database searches.

5.9 Items

The items table is the master catalogue for commodities. The unit field allows the system to express the measurement used for the item, while the item identifier is used by stock, entitlement and transaction records.

5.10 Stock

Stock connects shops and items and stores quantity_kg. This is a current-state table: unlike transactions, it represents what is presently available at the shop.

5.11 Relationship

A shop can hold many items, and an item can appear at many shops. The stock table therefore acts as the associative structure between the two entities.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Table	Primary Key	Important Foreign Keys	Purpose
shops	shop_id	owner_id	Identify Fair Price Shops
items	item_id	—	Commodity master list
stock	stock_id	shop_id, item_id	Current shop-item quantity

CHAPTER -5 DESIGN (CONTINUED)

5.12 Entitlement Records

Each entitlement belongs to a card and an item and carries month, year, quota_kg and collected_kg. The two quantity values make the remaining balance easy to determine.

5.13 Transaction Records

Transactions capture the historical event rather than the current balance. The card, shop, item, quantity and date form the core of the record.

5.14 Historical Versus Current Data

This distinction is important: stock is current state, entitlement stores the monthly allowance and consumption, while transactions provide the event history. Keeping these concepts separate makes reporting easier.

5.15 Reporting Possibilities

From these tables, an administrator can derive shop-wise issue history, card-wise consumption history and item-wise movement. These reports can be added without changing the fundamental role model.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Entity	Stores	Typical Question
Entitlements	Monthly allowance and collection	How much can this card still collect?
Transactions	Historical purchase events	What was purchased and when?
Stock	Current shop quantity	How much is available now?

CHAPTER -5 DESIGN (CONTINUED)

5.16 Grievance Record

A grievance links a card and shop with a subject, description, status and creation time. This creates enough context for an administrator to understand which transaction environment the complaint concerns.

5.17 Controlled Status

The status is limited to Pending and Resolved in the current design. Restricting the allowed values keeps the state machine simple and prevents arbitrary strings from appearing in reports.

5.18 Integrity Across Modules

Foreign keys connect grievances to valid cards and shops. Similar relationships exist in transactions and entitlements. Together, these constraints reduce the risk of orphaned records.

5.19 Future Audit Detail

A future production implementation could store who changed a grievance, when it was resolved and what resolution note was recorded. Such an audit trail would complement the current status field.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -5 DESIGN (CONTINUED)

5.20 Card Lookup

The most common lookup can search the unique card number. The existing index is intended to make this operation efficient as the number of cards grows.

5.21 Stock Lookup

A shop dashboard can join stock with items to display readable commodity names beside quantities. The join uses the item identifier rather than repeating the item name in every stock row.

5.22 Transaction History

A transaction-history query can filter by card or shop and order results by transaction date. The same underlying records can support different role-specific views.

5.23 Grievance Queue

An Admin dashboard can filter grievances by status to display pending complaints first. This is a simple example of turning stored state into an operational queue.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

```
SELECT r.card_no, e.item_id, e.month, e.year,
       e.quota_kg, e.collected_kg
FROM   ration_cards r
JOIN   entitlements e ON e.card_id = r.card_id
WHERE  r.card_no = ?
ORDER BY e.year DESC, e.month DESC;
```

CHAPTER -5 DESIGN (CONTINUED)

5.24 Authentication Dependency

Every protected module depends on authentication because the server needs to know which user is making the request.

5.25 Card Dependency

Entitlement and transaction operations depend on valid ration-card records.

5.26 Stock Dependency

The transaction module depends on current stock to determine whether a purchase can be completed.

5.27 Grievance Dependency

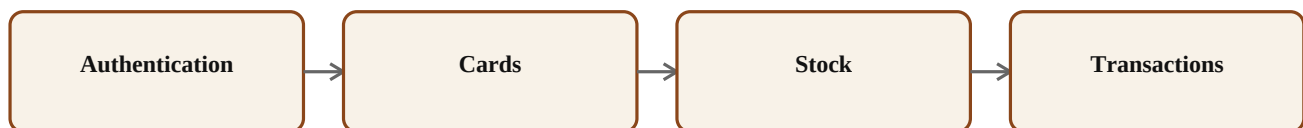
The grievance module depends on valid card and shop references so that complaints have meaningful context.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.



Illustrative workflow / component relationship

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

The application is implemented as five cooperating modules. Each module has its own routes in the back end and its own screens in the front end.

- Authentication and User Management module: registration, login and role checking; the Admin can create, update and remove users.
- Ration Card and Entitlement module: registers cards with a UNIQUE card number and generates the monthly quota for each item according to the household size and card type.
- Shop and Stock module: manages Fair Price Shops and their stock of each item; supports adding stock, correcting stock and viewing current levels.
- Transaction module: records a purchase against a card, verifies the remaining entitlement and the available stock, then updates both inside a single database transaction.
- Grievance module: lets a beneficiary submit a complaint and lets the Admin review it and mark it Resolved with remarks.

The core of the system is the transaction module, because it is where the entitlement, the stock and the history are kept consistent. The following SQL excerpt shows the design of the ration card table, including the UNIQUE constraint and the index used for fast lookup.

```
CREATE TABLE ration_cards (  
  card_id          INT AUTO_INCREMENT PRIMARY KEY,  
  card_no          VARCHAR(20) NOT NULL UNIQUE,  
  head_user_id     INT NOT NULL,  
  card_type        VARCHAR(20) NOT NULL,  
  members_count    INT NOT NULL CHECK (members_count > 0),  
  status           ENUM('Active','Inactive') DEFAULT 'Active',  
  FOREIGN KEY (head_user_id) REFERENCES users(user_id)  
);  
CREATE INDEX idx_card_no ON ration_cards(card_no);
```

6.2 Front-End Logic (UI Rendering, State Management)

Each dashboard is a set of HTML pages enhanced with JavaScript. When a page loads, the script reads the stored login token, calls the relevant API using the Fetch API and builds the tables and cards dynamically in the DOM. Forms are validated in the browser for empty and malformed values before submission, and the server-side validation acts as the final authority. Because the application is small, state is kept simply: the logged-in user's role and token are held in browser storage, and every other piece of information is fetched fresh from the server so that the display always reflects the current database.

Success and error messages are shown on the same page, so that a user entering a purchase or a complaint receives immediate feedback without leaving the screen.

6.3 API Integration

The front end communicates with the back end through REST endpoints that exchange JSON. The main endpoints are summarised below.

Method	Endpoint	Description	Role
POST	/api/auth/login	Authenticate a user and return a token	All
POST	/api/users	Create a user	Admin
POST	/api/cards	Register a ration card	Admin
GET	/api/cards/:cardNo	Fetch card details	Admin, Shop Owner, Beneficiary (own)
GET	/api/entitlements/:cardId	Monthly quota and collected quantity	Admin, Beneficiary (own)
GET	/api/stock/:shopId	Current stock of a shop	Admin, Shop Owner
PUT	/api/stock/:stockId	Update stock quantity	Admin, Shop Owner
POST	/api/transactions	Record a purchase and reduce stock	Shop Owner
GET	/api/transactions	Transaction history	Admin, Shop Owner, Beneficiary (own)
POST	/api/grievances	Submit a grievance	Beneficiary
PUT	/api/grievances/:id	Update status to Resolved	Admin
DELETE	/api/shops/:id	Remove a shop	Admin

The following excerpt illustrates how the transaction route keeps the entitlement, stock and history consistent by running all three updates inside one database transaction.

```
router.post('/transactions', auth('shop_owner'), async (req, res) => {
  const { cardId, shopId, itemId, quantity } = req.body;
  if (!cardId || !shopId || !itemId || quantity <= 0)
    return res.status(400).json({ error: 'Invalid input' });
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    // 1. check remaining entitlement    2. check shop stock
    // 3. insert transaction row        4. reduce stock, increase collected_kg
    await conn.commit();
    res.status(201).json({ message: 'Purchase recorded' });
  } catch (e) { await conn.rollback(); res.status(500).json({ error: e.message }); }
  finally { conn.release(); }
});
```

6.4 Authentication and Authorisation

Access to the system is protected in two steps. Authentication verifies who the user is: credentials are checked against the users table, passwords are stored only as hashes and never as plain text, and a successful login returns a token that must accompany every later request. Authorisation decides what the user may do: an

Express middleware reads the role from the token and rejects any request for which the role is not permitted with a 403 response.

Role-based access ensures that a beneficiary can see only the data of their own ration card, that a shop owner can change only the stock of their own shop, and that administrative actions such as creating shops or resolving grievances are limited to the Admin. All SQL statements use parameterised queries so that user input can never alter the structure of a query.

CHAPTER -6 IMPLEMENTATION (CONTINUED)

6.5 Login Endpoint

POST /api/auth/login accepts credentials and returns a token when authentication succeeds. The token becomes the credential used by subsequent protected requests. Passwords are not stored as plain text according to the project design.

6.6 User Creation

POST /api/users is an administrative operation. The request creates a user record with the selected role. The server should validate required fields and enforce the database uniqueness rule for email.

6.7 User Lifecycle

The Admin can manage user records as part of the CRUD model. A production implementation would also define rules for deactivation, password reset and audit history; these are natural extensions rather than claims about the current prototype.

6.8 Security Boundary

Authentication answers “who is this?” while authorisation answers “what can this user do?” Keeping these concepts separate prevents a successful login from becoming automatic permission to access every resource.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Method	Endpoint	Purpose	Role
POST	/api/auth/login	Authenticate and return token	All
POST	/api/users	Create user	Admin
GET	/api/users	Read user data	Admin
PUT	/api/users/:id	Update user	Admin
DELETE	/api/users/:id	Remove user	Admin

CHAPTER -6 IMPLEMENTATION (CONTINUED)

6.9 Card Registration

The card-registration operation creates a household record using a unique card number, card type, family head and member count. The unique constraint is an important database-level safeguard because it remains effective even if two requests reach the server unexpectedly close together.

6.10 Card Retrieval

GET /api/cards/:cardNo returns the card information required by an authorised role. A beneficiary should be restricted to the household associated with that account, while administrative users can access broader records.

6.11 Entitlement Retrieval

GET /api/entitlements/:cardId provides the monthly quota and collected quantity. The remaining quantity is conceptually the quota minus the collected amount, subject to the application's validation rules.

6.12 Monthly Records

The inclusion of month and year in the entitlement structure gives each period its own record. This avoids overwriting the history of previous months and allows a beneficiary or administrator to inspect consumption over time.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Method	Endpoint	Description	Role
POST	/api/cards	Register ration card	Admin
GET	/api/cards/:cardNo	Fetch card details	Admin / Shop Owner / Own Beneficiary
GET	/api/entitlements/:cardId	Monthly quota and collection	Admin / Own Beneficiary

CHAPTER -6 IMPLEMENTATION (CONTINUED)

6.13 Shop Records

A shop record contains a unique shop code, owner reference, name and location. The owner reference connects operational stock activity to the responsible Shop Owner.

6.14 Stock Records

Stock is represented as a relationship between a shop and an item. Keeping one row for a shop-item pair makes current inventory easy to read and update.

6.15 Stock Update

PUT /api/stock/:stockId updates the quantity when authorised. The server should ensure that the user has the required role and, for a Shop Owner, that the stock belongs to the owner's shop.

6.16 Operational Use

The stock module provides the state used by the transaction module. A purchase consumes stock, while an administrative restocking operation increases it. Keeping this value in the database allows the dashboard to show the current state rather than a manually calculated estimate.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Method	Endpoint	Description	Role
GET	/api/stock/:shopId	Current shop stock	Admin / Shop Owner
PUT	/api/stock/:stockId	Update stock quantity	Admin / Shop Owner
DELETE	/api/shops/:id	Remove a shop	Admin

CHAPTER -6 IMPLEMENTATION (CONTINUED)

6.17 Purchase Request

POST /api/transactions is the most important operational endpoint. The request contains cardId, shopId, itemId and quantity. The server validates each field before accessing the database.

6.18 Validation Sequence

The server checks that identifiers are present, the quantity is positive, the card exists, the shop exists, the item exists, the requested quantity does not exceed remaining entitlement and the shop has enough stock.

6.19 Atomic Update

The purchase is then written inside a database transaction. The transaction row records the event, stock decreases, and collected_kg increases. A failure anywhere causes rollback.

6.20 History

The resulting transaction row becomes immutable purchase history for reporting purposes. This history is visible according to role: beneficiaries see their own purchases, Shop Owners see their shop activity and Admin users can inspect the wider system.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

```
router.post('/transactions', auth('shop_owner'), async (req, res) => {
  const { cardId, shopId, itemId, quantity } = req.body;
  if (!cardId || !shopId || !itemId || quantity <= 0)
    return res.status(400).json({ error: 'Invalid input' });
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    // validate entitlement and stock
    // insert transaction; reduce stock; increase collected_kg
    await conn.commit();
    res.status(201).json({ message: 'Purchase recorded' });
  } catch (e) { await conn.rollback(); res.status(500).json({ error: e.message }); }
  finally { conn.release(); }
});
```

CHAPTER -6 IMPLEMENTATION (CONTINUED)

6.21 Submission

POST /api/grievances allows a beneficiary to submit a complaint with a subject and description. The record begins in Pending status.

6.22 Review

The Admin can retrieve pending grievances, inspect the associated card and shop and decide what action is required. The current prototype represents completion by changing the status to Resolved.

6.23 Status Visibility

The beneficiary does not need to repeatedly contact the shop or administration to ask about a complaint. The portal exposes the stored status so the user can see whether the issue remains pending or has been resolved.

6.24 Future Extension

A production version could add timestamps for status changes, resolution remarks, assignment to a department and a complete audit trail. These are proposed improvements rather than current implementation claims.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Method	Endpoint	Purpose	Role
POST	/api/grievances	Submit complaint	Beneficiary
GET	/api/grievances	Review complaints	Admin
PUT	/api/grievances/:id	Mark resolved	Admin

CHAPTER -6 IMPLEMENTATION (CONTINUED)

6.25 Client-Side Errors

Missing form values or clearly invalid formats should be identified before a request is sent. This reduces avoidable round trips and gives immediate feedback.

6.26 API Errors

The backend should distinguish invalid input, unauthorised access, missing records and unexpected server failures. The original testing section explicitly checks validation and role violations.

6.27 Database Errors

Database errors should be caught by the route handler. In multi-step operations, the connection must roll back before it is released. This prevents partially applied changes.

6.28 User-Facing Messages

Technical details such as stack traces should not be exposed to ordinary users. A concise message can explain what went wrong while detailed diagnostics remain available to the developer during testing.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -6 IMPLEMENTATION (CONTINUED)

6.29 Authentication

The project uses token-based authentication so later API requests can be associated with the logged-in user. Passwords are described as being stored as hashes rather than plain text.

6.30 Authorisation

Role-based authorisation is applied at the API layer. This protects administrative endpoints even when a user attempts to bypass the front-end interface.

6.31 Parameterised Queries

The report specifies parameterised SQL statements so user input cannot alter the structure of a query. This is an important protection against SQL injection.

6.32 Least Privilege

Each role receives only the permissions needed for its normal task. Limiting access reduces the impact of accidental misuse and makes the system easier to reason about.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -6 IMPLEMENTATION (CONTINUED)

6.33 Fetch Request

The browser uses the Fetch API to send JSON requests to the Express server. The request includes the authentication token for protected endpoints.

6.34 Response Handling

The JavaScript code checks the response, parses JSON and updates the relevant part of the page. A failed response should be displayed as a useful message rather than silently ignored.

6.35 Refreshing State

After a purchase or stock update, the dashboard should request fresh data so the displayed values correspond to the database state.

6.36 Separation

The front end does not connect directly to MySQL. This keeps database credentials and business rules on the server side.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

```
const response = await fetch('/api/transactions', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${token}`
  },
  body: JSON.stringify(payload)
});
const result = await response.json();
```

CHAPTER -6 IMPLEMENTATION (CONTINUED)

6.37 Begin

Open a database connection and start a transaction before changing related records.

6.38 Validate

Read the entitlement and stock values needed for the purchase decision. Reject the request if any condition fails.

6.39 Update

Insert the transaction and update collected_kg and quantity_kg as one logical unit.

6.40 Commit or Rollback

Commit only after every required operation succeeds. Roll back if an exception occurs, then release the connection.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

```
BEGIN TRANSACTION
validate card and item
read remaining entitlement
read shop stock
IF quantity is valid
INSERT transaction
UPDATE entitlement
UPDATE stock
ELSE
REJECT
COMMIT on success
ROLLBACK on failure
```

CHAPTER -7 FEATURES

7.1 List of Main Features

1. Digital ration cards with a UNIQUE card number.
2. Automatic monthly entitlement calculation and display.
3. Real-time Fair Price Shop inventory.
4. Purchase recording that synchronises with shop stock.
5. Transparent transaction history for every role.
6. Grievance submission with Pending / Resolved tracking.
7. Role-based dashboards for Admin, Shop Owner and Beneficiary.
8. Full CRUD operations on users, shops, stock and grievances.
9. Backend input validation and database-level integrity constraints.

7.2 How Each Feature Works

Digital ration card. The Admin registers a household by entering the card number, the head of the family, the card type and the number of members. The database rejects a card number that already exists, so a duplicate or ghost card cannot be created. The beneficiary can then log in and view the card details.

Monthly entitlement. For each month, the system stores a quota in kilograms for every item against the card. The beneficiary dashboard displays the quota, the quantity already collected and the balance, so the household knows exactly what it may still collect.

Real-time stock and purchase recording. When a beneficiary arrives at the shop, the shop owner enters the card number, item and quantity. The server confirms that the quantity does not exceed the balance entitlement and that the shop has enough stock. If both checks pass, the transaction is stored, the stock is reduced and the collected quantity is increased together, so the Admin immediately sees the updated stock level.

Transaction history. Every purchase is stored with the card, shop, item, quantity and date. The beneficiary sees only their own purchases, the shop owner sees the purchases made at their shop, and the Admin sees all of them, which makes irregular patterns easy to spot.

Grievance redressal. A beneficiary submits a complaint with a subject and description, which is stored with the status Pending. The Admin reviews the complaint, takes action and marks it Resolved with remarks. The beneficiary can follow the status at any time instead of waiting without information.

Role-based dashboards and CRUD. Each role sees only the screens and actions that apply to it. The Admin can create, view, update and delete users and shops, the shop owner can create and update stock entries, and the beneficiary can create grievances and read their own data.

CHAPTER -7 FEATURES (CONTINUED)

7.3 Duplicate Registration

A duplicate ration card could distort entitlement allocation. The combination of server validation and a UNIQUE database constraint provides two layers of protection.

7.4 Unauthorised Data Access

A beneficiary should not be able to request another household's history simply by changing an identifier in the URL. The server must check ownership rather than relying on the browser to hide records.

7.5 Excessive Purchase Quantity

A malicious or accidental request could attempt to collect more than the remaining entitlement or available stock. The transaction module checks both values before committing.

7.6 Credential Protection

Credentials should be transmitted over HTTPS in a public deployment and passwords should remain hashed. Secrets such as database credentials and JWT secrets belong in environment configuration rather than source control.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -7 FEATURES (CONTINUED)

7.7 Assets

Important assets include user credentials, ration-card details, entitlement records, stock values, transaction history and grievance information. Protecting these records preserves both privacy and data integrity.

7.8 Trust Boundaries

The browser is an untrusted boundary. The user can modify requests manually, so all security-sensitive checks must occur in the server and database layers.

7.9 Abuse Cases

Potential abuse cases include attempting to access another beneficiary's records, manipulating purchase quantities, invoking administrative endpoints, or submitting malformed values. The current architecture addresses these through authentication, authorisation, validation and constraints.

7.10 Monitoring

A production deployment would benefit from structured server logs for failed authentication, rejected transactions and administrative changes. Monitoring is a future operational enhancement, not a claim about the current prototype.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -7 FEATURES (CONTINUED)

7.11 Login

Open the application login page and sign in with the Admin account. The server determines the role and the application loads the corresponding dashboard.

7.12 Manage Users and Shops

Use the administrative screens to create and maintain users and Fair Price Shops. Shop ownership should be assigned carefully because it controls which Shop Owner can update the shop's operational data.

7.13 Register Cards and Stock

Register ration cards with their unique card numbers and maintain shop stock for the available items. Verify the values after entry so later transactions use correct starting data.

7.14 Monitor and Resolve

Use transaction history to inspect purchases and the grievance queue to review Pending complaints. Resolve a grievance only after the required action has been completed.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -7 FEATURES (CONTINUED)

7.15 Login and Stock

Sign in as a Shop Owner and review the stock assigned to the shop. If the system permits an authorised stock correction, use the stock update operation rather than changing database records manually.

7.16 Record a Purchase

Enter the card number, item and quantity requested. The server verifies the entitlement and available stock before saving the transaction.

7.17 Confirm the Result

After a successful purchase, the system returns a confirmation. The stock quantity and transaction history should reflect the operation immediately.

7.18 Handling Rejection

If a purchase is rejected because stock or entitlement is insufficient, do not attempt to bypass the validation. Review the displayed values and correct the request or escalate the issue to the Admin.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -7 FEATURES (CONTINUED)

7.19 View Card

Sign in to view the digital ration card information associated with the household. The interface should display the card identity without exposing unrelated households.

7.20 Check Entitlement

Review the monthly quota, collected quantity and remaining balance for each item. This provides a direct view of what has been recorded against the card.

7.21 Review History

Open transaction history to see previous purchases, including the item, quantity, shop and date as supported by the transaction record.

7.22 Submit a Grievance

If a problem needs administrative attention, submit a grievance with a clear subject and description. The record begins as Pending and can later be checked for its updated status.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -7 FEATURES (CONTINUED)

7.23 Setup

Create one Admin account, one Shop Owner, one Beneficiary, one shop, a small set of items, a ration card and initial stock. This produces a compact environment suitable for a classroom demonstration.

7.24 Demonstration Sequence

Begin with Admin registration of the shop and card, then add stock. Switch to the Shop Owner dashboard and record a purchase. Finally switch to the Beneficiary dashboard to show the updated entitlement and transaction history.

7.25 Grievance Demonstration

Submit a grievance as the beneficiary, display it in the Admin queue, mark it Resolved and return to the beneficiary view. This demonstrates the second major end-to-end workflow.

7.26 Evidence

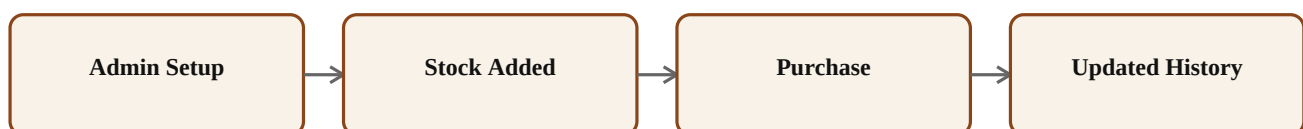
Screenshots of each stage can be inserted in the appendix. The screenshots should show the application state and avoid revealing any real personal data.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.



Illustrative workflow / component relationship

CHAPTER -7 FEATURES (CONTINUED)

7.27 Purpose

The following records are illustrative synthetic examples that show how the entities described in the report can be represented during a demonstration. They are not real beneficiary records.

7.28 User Examples

A small dataset can contain one Admin, one Shop Owner and one Beneficiary. Each account has a distinct role so that the role-based dashboard behaviour can be demonstrated.

7.29 Card Examples

Sample card numbers should be unique and clearly fictional. The associated member count and card type can be varied to demonstrate validation and entitlement display.

7.30 Shop and Item Examples

A demonstration can use multiple shops and a small commodity list. This is enough to exercise stock relationships and transaction history without requiring a large database.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Entity	Example Value	Purpose
User	admin.demo@example.test	Admin login example
Card	DEMO-10001	Synthetic card identifier
Shop	FPS-DEMO-01	Synthetic shop code
Item	Rice	Commodity example

CHAPTER -7 FEATURES (CONTINUED)

7.31 Full Balance

A newly created monthly entitlement can begin with collected_kg equal to zero. The displayed balance is therefore equal to the quota.

7.32 Partial Collection

After a successful purchase, collected_kg increases and the remaining balance decreases. This demonstrates why the entitlement record stores both values.

7.33 Exact Collection

A purchase equal to the remaining balance should bring the collected value to the quota. Further purchases for the same period should be rejected by the entitlement rule.

7.34 Historical Period

A previous month remains stored as a separate record. This makes it possible to inspect earlier consumption without changing the current month.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Scenario	Quota	Collected	Concept
New month	10 kg	0 kg	Full balance
Partial issue	10 kg	4 kg	6 kg remaining
Exact completion	10 kg	10 kg	No balance remaining
Previous period	Stored	Stored	Historical record

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Postman was used to verify the REST APIs independently of the user interface. A collection of requests was prepared for every endpoint, covering the four HTTP verbs used by the application: GET for reading data, POST for creating records, PUT for updating records and DELETE for removing them. Each request was tested with valid input to confirm the expected response and with invalid input to confirm that the server returned a suitable error.

Typical negative tests included registering a ration card with a card number that already exists, recording a purchase that exceeds the available stock, submitting a request with missing fields, and calling an endpoint with a role that is not permitted. In each case the API responded with the correct status code and an explanatory message, and the database remained unchanged.

8.2 Integration Testing

After the individual APIs were verified, the front end and back end were tested together. The main flow tested end to end was: the Admin registers a shop and a ration card, the Admin adds stock to the shop, the shop owner records a purchase against the card, the beneficiary views the updated entitlement and history, and the Admin sees the reduced stock. The results confirmed that cards are digitised and that purchases synchronise with shop stock in real time.

A second flow tested the grievance cycle: a beneficiary submits a complaint, the Admin sees it as Pending, resolves it, and the beneficiary then sees the status as Resolved.

8.3 Tools Used for Testing

Tool	Purpose
Postman	Sending requests to REST APIs and checking responses and status codes
Browser Developer Tools	Inspecting network calls, console errors and page rendering
MySQL Workbench	Verifying that data was stored, updated or rejected correctly in the database
Git and GitHub	Tracking changes and re-testing after each merge

8.4 Test Cases and Results

ID	Test Case	Expected Result	Result
TC-01	Register a ration card with a new card number	Card is created and stored	Pass
TC-02	Register a card with an existing card number	Request rejected (UNIQUE constraint)	Pass
TC-03	Login with valid credentials	Token returned; correct dashboard opens	Pass
TC-04	Access an Admin endpoint as a beneficiary	Access denied (403)	Pass
TC-05	Record a purchase within entitlement and stock	Transaction saved; stock reduced	Pass

ID	Test Case	Expected Result	Result
TC-06	Record a purchase larger than available stock	Request rejected; stock unchanged	Pass
TC-07	Submit a request with missing or invalid fields	Validation error (400)	Pass
TC-08	Create, read, update and delete a shop	All four operations work	Pass
TC-09	Submit and resolve a grievance	Status changes from Pending to Resolved	Pass
TC-10	Search a card by card number	Result returned using the index	Pass

All the listed test cases produced the expected results. The testing confirms that the cards are digitised, that purchases synchronise with shop stock in real time, that full CRUD works for users, shops, stock and grievances, that the card number is unique, and that the APIs behave correctly when verified in Postman.

CHAPTER -8 TESTING (CONTINUED)

8.5 Unit-Level Thinking

Individual routes and helper functions can be tested with controlled inputs. The objective is to verify that validation, permission checks and database calls behave correctly for expected and invalid values.

8.6 API Testing

Postman provides a direct way to test REST endpoints without depending on the browser. The original report uses this approach for both positive and negative cases.

8.7 Integration Testing

Integration testing verifies that separate modules work together. The project's main end-to-end path connects Admin setup, stock creation, Shop Owner purchase, beneficiary history and Admin stock visibility.

8.8 Regression Testing

After changing a route or database query, previously successful test cases should be repeated. Git branches and commits make this iterative process easier to manage.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -8 TESTING (CONTINUED)

8.9 Test Philosophy

Authentication tests should cover both successful use and incorrect credentials. The goal is to verify that valid users can reach their intended dashboard while invalid credentials do not receive a valid session token.

8.10 Boundary Cases

Empty fields, incorrect password, unknown email and malformed request bodies are useful cases. The expected response should be an error without exposing sensitive information.

8.11 Role Verification

A successful login must produce a role associated with the correct account. The subsequent authorisation layer should then use that role when deciding which routes are accessible.

8.12 Test Evidence

For a formal submission, screenshots of representative Postman requests and response status codes can be placed in the appendix. This provides visual evidence without changing the application logic.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

ID	Scenario	Expected Result
AUTH-01	Valid credentials	Token returned
AUTH-02	Wrong password	Authentication rejected
AUTH-03	Unknown account	Authentication rejected
AUTH-04	Missing fields	Validation error
AUTH-05	Beneficiary calls Admin API	403 / denied

CHAPTER -8 TESTING (CONTINUED)

8.13 Creation Tests

A valid card should be stored. A duplicate card number should be rejected. A missing head user or invalid member count should not create a record.

8.14 Retrieval Tests

Searching by a valid card number should return the expected card. Searching for a non-existent number should produce a controlled not-found response.

8.15 Ownership Tests

A beneficiary should retrieve only the card associated with the account. A Shop Owner may retrieve a card for operational purchase verification, according to the role permissions in the report.

8.16 Database Verification

After each test, MySQL Workbench can be used to verify whether the expected row was created or rejected. This confirms that a successful API response corresponds to actual persistence.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

ID	Scenario	Expected Result
CARD-01	New unique card	Created
CARD-02	Duplicate card number	Rejected
CARD-03	Invalid member count	Rejected
CARD-04	Existing card lookup	Details returned
CARD-05	Unknown card lookup	Not found

CHAPTER -8 TESTING (CONTINUED)

8.17 Stock Creation

Adding a valid shop-item quantity should create or update the corresponding stock record. The quantity must be represented consistently with the project's kilogram-based fields.

8.18 Stock Reduction

A successful purchase must reduce stock by exactly the purchased amount. No other item or shop should be affected.

8.19 Insufficient Stock

A purchase larger than available stock must be rejected and the original stock value must remain unchanged. This is one of the explicit negative cases in the original report.

8.20 Ownership

A Shop Owner should not be able to update a different owner's shop stock. This must be checked server-side.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

ID	Scenario	Expected Result
STOCK-01	Valid stock update	Quantity updated
STOCK-02	Purchase within stock	Stock reduced
STOCK-03	Purchase exceeds stock	Rejected; unchanged
STOCK-04	Wrong shop owner	Access denied
STOCK-05	Invalid quantity	Validation error

CHAPTER -8 TESTING (CONTINUED)

8.21 Monthly Quota

An entitlement record should expose the quota and collected quantity for the selected month and item.

8.22 Balance Calculation

The remaining quantity should never be treated as greater than the quota. A purchase updates collected_kg so the displayed balance changes immediately after a successful transaction.

8.23 Over-Collection

A purchase request that exceeds the remaining entitlement must be rejected before any stock or transaction changes are committed.

8.24 Historical Periods

Previous months should remain stored rather than being overwritten. This allows later reporting and provides a trace of earlier entitlement periods.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

ID	Scenario	Expected Result
ENT-01	Read current month	Quota and collection shown
ENT-02	Valid purchase	Collection increases
ENT-03	Exceeds remaining quota	Rejected
ENT-04	Previous month query	Historical record retained

CHAPTER -8 TESTING (CONTINUED)

8.25 Submission

A beneficiary should be able to submit a complaint with the required subject and description. The new record starts as Pending.

8.26 Administrative Resolution

The Admin can review the complaint and change its state to Resolved. The beneficiary can then see the updated state.

8.27 Invalid Input

Missing subject or description should be rejected with a validation response. The database should not contain a partial grievance.

8.28 Visibility

A beneficiary should see only the grievances associated with the beneficiary's own card, while the Admin can inspect the wider grievance queue.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

ID	Scenario	Expected Result
GRV-01	Submit valid grievance	Pending record created
GRV-02	Missing subject	Rejected
GRV-03	Admin resolves	Status becomes Resolved
GRV-04	Beneficiary checks status	Own status displayed

CHAPTER -8 TESTING (CONTINUED)

8.29 Purpose

Test data should represent realistic relationships without depending on real beneficiary information. Synthetic names, card numbers and shop codes are appropriate for an academic prototype.

8.30 Data Variety

A useful test set contains more than one role, several shops, several items, multiple ration cards and transactions across different dates. This makes list views and filters more representative.

8.31 Boundary Values

Testing should include one-member households, larger member counts, zero or negative quantities, exact-entitlement purchases and quantities just above the entitlement.

8.32 Reproducibility

Keeping a small seed dataset or documented insert sequence makes it easier to repeat tests after a database reset. This improves consistency during demonstrations and grading.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -8 TESTING (CONTINUED)

8.33 Response Time

For a prototype, basic responsiveness can be observed through browser developer tools and Postman. The important point is to identify unusually slow endpoints rather than claim a specific benchmark that has not been measured.

8.34 Database Queries

Queries that search card numbers, shop records or recent transactions should be examined for appropriate indexes. MySQL query plans can be used if deeper optimisation becomes necessary.

8.35 Concurrent Purchases

A production PDS environment could have multiple transactions arriving close together. Database transactions and appropriate locking or isolation behaviour would need to be evaluated before real deployment.

8.36 Scalability

The current architecture is suitable for a prototype and classroom demonstration. Larger deployments would require capacity planning, database monitoring, connection-pool tuning and possibly horizontal scaling.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

1. Install Node.js, npm and MySQL on the target machine.
2. Clone the project repository from GitHub.
3. Create the database and run the SQL script to create all tables, constraints and indexes.
4. Create a .env file containing the database credentials, the server port and the secret key.
5. Run npm install to download the required packages.
6. Start the server with npm start (or node server.js).
7. Open the application in a browser at the configured address and log in as the Admin.

9.2 Environment Configuration

Sensitive settings are kept outside the source code in an environment file, which is excluded from version control. A sample configuration is shown below.

```
PORT=3000
DB_HOST=localhost
DB_USER=pds_user
DB_PASSWORD=<your_password>
DB_NAME=pds_tracker
JWT_SECRET=<long_random_string>
```

Separate files can be maintained for development and production so that the same code runs in both environments with different database credentials.

9.3 Hosting of Frontend and Backend

Because Express serves the front-end files as static assets, the whole application can be hosted as a single Node.js service. For a classroom demonstration, running it on a local machine is sufficient. For wider access, the Node.js service can be placed on a free-tier cloud platform and connected to a managed MySQL database; the free tier is enough for a prototype and keeps the cost of the project at zero. HTTPS should be enabled on the public deployment so that credentials are encrypted in transit.

CHAPTER -9 DEPLOYMENT (CONTINUED)

9.4 Local Components

The original development arrangement places the Express server, MySQL service and front-end files on the same machine. This is suitable for classroom demonstrations because it avoids external hosting dependencies.

9.5 Startup Sequence

The database service must be available before the server attempts database connections. After dependencies are installed and environment variables are configured, the Node.js server can be started and the browser can open the configured address.

9.6 Local Verification

A successful local deployment should be checked by opening the login page, authenticating a test account, performing a small database-backed operation and verifying the corresponding record in MySQL.

9.7 Recovery

If the application fails locally, the developer should check the server console, database status, environment variables, port conflicts and package installation before changing application code.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -9 DEPLOYMENT (CONTINUED)

9.8 Production-Style Layout

The report proposes a free-tier Node.js service connected to a managed MySQL instance. The front end can continue to be served by Express as static assets, preserving the three-layer logical architecture.

9.9 Environment Variables

Cloud deployment requires database host, user, password, database name, server port and secret values to be configured through the hosting platform rather than committed to the repository.

9.10 HTTPS

The original deployment section recommends HTTPS for public access so credentials are encrypted in transit. This is an essential requirement before exposing login functionality on a public network.

9.11 Operational Readiness

A production deployment would additionally require backups, logging, monitoring, dependency updates and a documented recovery process. These are operational concerns beyond the current classroom prototype.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -9 DEPLOYMENT (CONTINUED)

9.12 Why Backups Matter

The database contains the system's authoritative records. Losing the database would affect cards, entitlements, transactions, stock and grievances, so backup planning is essential for a real deployment.

9.13 Backup Frequency

A production schedule should reflect how frequently the organisation can tolerate losing data. The classroom prototype does not specify a live backup policy, so this section treats it as a deployment recommendation.

9.14 Restore Verification

A backup is only useful if it can be restored. Periodic restoration tests should verify that the SQL dump or managed backup actually recreates the required tables and relationships.

9.15 Recovery Documentation

Recovery steps should include database restoration, environment configuration, application restart and a basic end-to-end verification. This turns backup files into an actionable recovery plan.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -9 DEPLOYMENT (CONTINUED)

9.16 Application Logs

Server logs can record startup, request errors, authentication failures and unexpected exceptions. Logs should avoid storing passwords or unnecessary personal information.

9.17 Business Events

Important operational events such as completed transactions and grievance resolution can be represented in database records. These records are distinct from technical logs because they describe application activity.

9.18 Health Checks

A lightweight health endpoint could report whether the server is running and whether a database connection is available. Such an endpoint can support deployment troubleshooting.

9.19 Monitoring Future

A larger deployment could add dashboards for error rates, response times and database health. This is a future operational enhancement rather than a current feature claim.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -9 DEPLOYMENT (CONTINUED)

9.20 Login Failure

If login fails, first verify the test account credentials and confirm that the database contains the expected user record. Next check that the server is running and that the browser is sending the request to the correct address.

9.21 401 Responses

A 401-style response generally indicates an authentication problem. Check whether the token was returned, stored and attached to subsequent requests.

9.22 403 Responses

A 403 response in the project's testing examples indicates that the role is not permitted to access the requested operation. Verify that the logged-in account has the intended role.

9.23 400 Responses

Validation errors should be investigated by comparing the request body with the endpoint requirements. Missing identifiers or non-positive quantities are common examples for the transaction endpoint.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -9 DEPLOYMENT (CONTINUED)

9.24 Connection Problems

Check the database service, host, port, username, password and database name configured in the environment. A server that starts but cannot obtain a connection will fail when a database-backed route is called.

9.25 Missing Tables

If the application reports missing tables, the schema creation script may not have been executed against the configured database. Verify the active database in MySQL Workbench.

9.26 Foreign Key Errors

A foreign key error usually means a referenced parent record does not exist or an operation would create an invalid relationship. Check the identifiers supplied by the API request.

9.27 Duplicate Errors

A duplicate card number or email can trigger a UNIQUE constraint. The correct response is to use a different value or update the existing record through the authorised workflow.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Keeping stock, entitlement and transaction data consistent when a purchase updates several tables at once.
- Preventing duplicate or invalid ration cards without blocking genuine registrations.
- Designing role-based access so that each user sees only the data they are permitted to see.
- Coordinating front-end pages with the back-end API as both were being developed in parallel.
- Presenting entitlement information in a simple form for users with limited technical experience.

10.2 Solutions Applied

- Database transactions (commit and rollback) were used so that all related updates succeed or fail together.
- A UNIQUE constraint on the card number and server-side validation were added to stop duplicate cards.
- Authorisation middleware was written once and applied to every protected route.
- The API contract was agreed first and tested in Postman, which allowed the front end to be built against stable endpoints.
- The dashboards were kept minimal, with clear labels and a balance display instead of complex tables.

10.3 Current Limitations or Known Bugs

- The system does not include Point-of-Sale hardware; the shop owner enters purchases manually.
- Identity at the counter is checked through the card number only; there is no biometric verification yet.
- There are no automatic SMS or e-mail notifications.
- The application has not been connected to any official government database and works on data entered into it.
- The interface is currently available in one language.

CHAPTER -10 CHALLENGES & LIMITATIONS (CONTINUED)

10.4 Blank Dashboard

A blank dashboard may result from a failed API request or a JavaScript error. Browser developer tools can reveal console errors and network responses.

10.5 Stale Display

Because dashboard data is fetched from the server, a stale value may indicate that the page has not re-fetched data after an operation or that the request failed. Inspect the network call before changing database records.

10.6 Form Problems

If a form refuses to submit, inspect client-side validation and confirm that required fields have values in the expected format.

10.7 Browser Compatibility

The report lists current Chrome, Edge and Firefox versions as suitable browsers. Testing across more than one browser is useful for identifying layout or JavaScript differences.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -10 CHALLENGES & LIMITATIONS (CONTINUED)

10.8 Opening Stock

The stock table represents the quantity currently held by a shop. A demonstration should begin by entering a known quantity so later changes can be verified.

10.9 Purchase Movement

A successful purchase reduces stock by the purchased quantity. The reduction should be equal to the transaction quantity and should affect only the selected shop-item pair.

10.10 Restocking

An authorised stock update can increase the quantity when new supply is received. The updated value becomes the new current state.

10.11 Reconciliation

The transaction history provides the event record that explains why stock changed. This separation between current stock and historical transactions is useful during reconciliation.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Step	Opening	Movement	Closing
Initial	50 kg	—	50 kg
Purchase	50 kg	−5 kg	45 kg
Restock	45 kg	+20 kg	65 kg
Purchase	65 kg	−10 kg	55 kg

CHAPTER -10 CHALLENGES & LIMITATIONS (CONTINUED)

10.12 Historical View

Transaction history records the card, shop, item, quantity and date. It is the audit-friendly record of completed purchases in the prototype.

10.13 Beneficiary View

A beneficiary should see only transactions associated with the household. The purpose is to provide transparency without exposing unrelated data.

10.14 Shop View

The Shop Owner can inspect activity belonging to the shop. This is useful for checking recent issues and comparing them with current stock.

10.15 Admin View

The Admin can inspect the broader history to identify unusual patterns or investigate a grievance. The current prototype provides visibility; advanced analytics remain future work.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Date	Card	Shop	Item	Qty
Demo date 1	DEMO-10001	FPS-DEMO-01	Rice	5 kg
Demo date 2	DEMO-10002	FPS-DEMO-01	Wheat	3 kg
Demo date 3	DEMO-10001	FPS-DEMO-02	Sugar	2 kg

CHAPTER -10 CHALLENGES & LIMITATIONS (CONTINUED)

10.16 Pending State

A newly submitted grievance begins as Pending. This gives the administrator a clear queue of unresolved issues.

10.17 Resolution State

Once reviewed, the Admin can change the state to Resolved. The beneficiary can then see the updated status.

10.18 Record Context

The card and shop references provide context for the complaint. A future implementation can add resolution notes and status timestamps.

10.19 Demonstration Use

A pair of synthetic grievances—one Pending and one Resolved—makes the dashboard state indicators easy to demonstrate during evaluation.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

ID	Subject	Status	Role
GRV-DEMO-01	Stock discrepancy	Pending	Beneficiary
GRV-DEMO-02	Entitlement query	Resolved	Beneficiary

CHAPTER -10 CHALLENGES & LIMITATIONS (CONTINUED)

10.20 User Creation

A user-creation request should include the fields required by the users table and a valid role. Sensitive password values should never be written into documentation as real credentials.

10.21 Card Creation

A card-creation request needs the card number, head user, card type and member count. The server and database enforce uniqueness and validity.

10.22 Transaction Creation

The transaction body contains cardId, shopId, itemId and quantity. The server performs all cross-entity checks before committing.

10.23 Grievance Creation

A grievance body contains the card and shop context together with a subject and description. The status is initially controlled by the server.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

```
{
  "cardId": 101,
  "shopId": 12,
  "itemId": 3,
  "quantity": 5
}
```

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

- SMS and e-mail alerts when the monthly quota reaches the local Fair Price Shop, so beneficiaries know when to collect their ration.
- Biometric authentication (fingerprint or Aadhaar) at the time of transaction to eradicate identity fraud.
- Multilingual support so that beneficiaries can use the portal in their local language.
- Low-stock alerts to the Admin when a shop is about to run out of an item.

11.2 Possible Integrations or Optimisations

- Integration with Point-of-Sale devices and weighing scales so that quantities are captured automatically.
- Analytics dashboards for administrators showing consumption trends, shop-wise distribution and unusual activity.
- A mobile application or Progressive Web App for beneficiaries.
- Integration with official beneficiary databases for automatic verification of cards.
- Database optimisations such as partitioning of the transactions table and caching of frequent queries as the volume of data grows.

CHAPTER -11 FUTURE ENHANCEMENTS (CONTINUED)

11.3 Traceability Method

Each major objective in the original report can be linked to one or more modules and tests. This provides a simple way to demonstrate that the project objectives are reflected in implementation.

11.4 Transparency

Digital card details, entitlement display and transaction history map directly to the Beneficiary dashboard and database records.

11.5 Stock Visibility

Real-time stock updates map to the Shop and Stock module and the transaction consistency mechanism.

11.6 Grievance Handling

The grievance objective maps to submission, administrative resolution and status visibility.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

Objective	Primary Module	Verification
Digitise cards	Ration Cards	Card creation/search tests
Track stock	Stock + Transactions	Stock movement tests
Transparency	Beneficiary Dashboard	History/entitlement checks
Grievances	Grievance Module	Pending/Resolved tests

CHAPTER -11 FUTURE ENHANCEMENTS (CONTINUED)

11.7 SMS and Email

The original report proposes SMS and e-mail alerts. Such notifications could inform beneficiaries about availability or other system events without changing the core database model.

11.8 Event Trigger

A notification service would listen for defined events, such as quota availability or administrative resolution, and send a message through an external provider.

11.9 Reliability

A production notification layer should handle provider failures and avoid sending duplicate messages. These concerns are beyond the current prototype.

11.10 Privacy

Messages should contain only the minimum necessary information. Sensitive household data should not be unnecessarily included in notifications.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -11 FUTURE ENHANCEMENTS (CONTINUED)

11.11 Shop Analytics

Administrators could view issue volumes by shop and period to support operational review.

11.12 Item Analytics

Item-wise movement could reveal which commodities are being distributed most frequently within the recorded dataset.

11.13 Anomaly Indicators

Future analytics could flag unusually high or low activity for human review. Such indicators would assist investigation rather than automatically declaring wrongdoing.

11.14 Data Visualisation

Charts and summary cards could present trends while detailed transaction tables remain available for verification.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -11 FUTURE ENHANCEMENTS (CONTINUED)

11.15 Motivation

A Progressive Web App could provide a more convenient beneficiary experience on smartphones while reusing much of the existing web architecture.

11.16 Offline Consideration

Some informational pages could potentially be cached, but transaction operations should require current server state so stock and entitlement remain consistent.

11.17 Responsive Reuse

The existing HTML, CSS and JavaScript approach provides a straightforward base for responsive layouts without requiring a separate native application.

11.18 Deployment

The PWA layer would remain connected to the same REST API and database, so the backend integrity rules would continue to apply.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -11 FUTURE ENHANCEMENTS (CONTINUED)

11.19 Point-of-Sale Device

A POS terminal could capture the card and transaction details directly, reducing manual typing at the shop.

11.20 Digital Weighing Scale

A connected scale could provide the actual measured quantity to the transaction process. The server would still validate the resulting quantity against entitlement and stock.

11.21 Integration Boundary

The hardware layer should communicate with a controlled interface rather than directly modifying the database. The existing API can remain the business-rule boundary.

11.22 Reliability

Hardware disconnection, incorrect readings and calibration issues would need dedicated error handling and audit records.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -11 FUTURE ENHANCEMENTS (CONTINUED)

11.23 Interface Strings

Text shown by the interface can be stored in language resource objects rather than embedded throughout JavaScript and HTML.

11.24 Language Selection

A user preference could determine which language resource is loaded. The underlying API data model does not need to change for basic translation.

11.25 Localisation Scope

Translation should cover navigation, form labels, status messages, validation messages and user guidance rather than only headings.

11.26 Testing

Each supported language should be checked for text overflow, correct meaning and consistent terminology.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -11 FUTURE ENHANCEMENTS (CONTINUED)

11.27 Current State

The present prototype identifies a household using the ration card number. This is intentionally simpler than a live identity-verification system.

11.28 Biometric Direction

The original report proposes fingerprint or Aadhaar-related authentication as a future enhancement. Any such integration would require appropriate government-approved interfaces and security controls.

11.29 Privacy Considerations

Identity data should be minimised and protected. A future integration must define exactly which information is stored locally and which remains with the external identity service.

11.30 Failure Handling

A production identity layer would need clear handling for unavailable devices, failed matches and network interruptions so that users are not left without a controlled outcome.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The Digital Ration Card & PDS Tracker is a robust full-stack web application that digitises the Public Distribution System into a secure, transparent and user-friendly portal. It replaces paper records with a relational database, gives administrators real-time visibility of shop stock to help prevent fraud, and lets beneficiaries view their entitlements and purchase history. The system was developed using HTML5, CSS3 and Vanilla JavaScript on the front end, Node.js and Express.js on the back end, and MySQL as the database.

12.2 What Was Achieved

- Ration cards were digitised, with a UNIQUE card number enforced and backend input validation applied.
- Purchases synchronise with shop stock in real time.
- Full CRUD operations were implemented for users, shops, stock and grievances.
- Role-based dashboards were created for the Admin, the Shop Owner and the Beneficiary.
- Transparent transaction history and a Pending / Resolved grievance tracker were provided.
- SQL was indexed for fast lookup, and the APIs were verified in Postman.

12.3 Skills Learned During Development

The project gave the team practical experience in relational database design — normalisation, keys, constraints, indexing and transactions — and in building REST APIs with Node.js and Express.js. The team learned to connect a vanilla JavaScript front end to a back end, to apply role-based access control, to validate input on both the client and the server, and to test APIs systematically with Postman. Working together through Git and GitHub improved collaboration, version control and task planning, and the work on a socially relevant problem gave a deeper understanding of how technology can improve transparency in public services.

CHAPTER -12 CONCLUSION (CONTINUED)

12.4 Objective One

Digitising ration cards and storing monthly entitlements creates a consistent digital representation of household eligibility within the prototype.

12.5 Objective Two

Recording current shop stock and synchronising it with purchases gives the administrator a more immediate view than a periodic manual register.

12.6 Objective Three

Transaction history and grievance tracking provide beneficiaries with visibility into recorded activity and complaint status.

12.7 Objective Four

Unique constraints, foreign keys, indexes, validation and transactions collectively protect the integrity of stored records.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -12 CONCLUSION (CONTINUED)

12.8 No POS Hardware

The current system relies on manual entry by the Shop Owner. This keeps the prototype simple but leaves room for input errors and does not automate weighing.

12.9 No Biometrics

Identity is represented through the card number in the prototype. Biometric verification is explicitly listed as a future enhancement.

12.10 No Government Database

The application operates on its own entered data. It should not be described as a live government PDS integration.

12.11 Single-Language Interface

The current interface is single-language. Localisation is a future enhancement that can improve accessibility.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -12 CONCLUSION (CONTINUED)

12.12 Database Engineering

The project provides practical experience with entity modelling, keys, constraints, indexing, joins and transactions.

12.13 Backend Development

Building Express routes demonstrates request handling, middleware, validation, JSON responses and integration with a relational database.

12.14 Front-End Development

The team gains experience in HTML structure, CSS layout, JavaScript events, Fetch API calls and dynamic DOM rendering.

12.15 Software Engineering

Git branching, Postman testing, modular design and documentation connect individual programming concepts to a complete engineering workflow.

Additional Implementation Note

The same design principles continue throughout this module: the browser is treated as a presentation layer, the Express API acts as the validation and business-logic boundary, and MySQL remains the persistent source of structured records. Keeping these responsibilities separate makes the project easier to understand, test and extend without changing the basic architecture.

Evaluation Note

For project review, the important evidence is the relationship between the documented requirement, the implemented workflow and the corresponding database or API behaviour. Screenshots, Postman responses and database observations can be used alongside the narrative where actual project evidence is required. The discussion on this page is intended to explain the implementation rather than introduce a new feature claim.

CHAPTER -13 REFERENCES

- [1] Node.js Documentation — <https://nodejs.org>
- [2] Express.js — <https://expressjs.com>
- [3] MySQL Reference Manual — <https://dev.mysql.com/doc>
- [4] Postman Learning Center — <https://learning.postman.com>
- [5] Git Documentation — <https://git-scm.com/doc>
- [6] MDN Web Docs (HTML, CSS, JavaScript) — <https://developer.mozilla.org>
- [7] National Food Security Act, 2013 — Government of India.

CHAPTER -14 APPENDICES

Screenshots of the App

Screenshot 1 — Login Page and Beneficiary Dashboard (ration card, monthly entitlement and history)

[paste screenshot here]

Screenshot 2 — Admin / Shop Owner Panel: Real-time Stock & Transaction Logs

[paste screenshot here]

Screenshot 3 — Grievance Redressal Screen (Pending / Resolved)

[paste screenshot here]

Sample Code Snippets

Representative code for the database schema and the transaction API is given in Sections 6.1 and 6.3 of this report. The complete source code is available in the project's GitHub repository.

Installation / Setup Instructions

The step-by-step installation procedure and the sample environment file are given in Sections 9.1 and 9.2.

User Manual or Guide

- Admin: log in, add shops and users, register ration cards, add stock and monitor transactions and grievances.
- Shop Owner: log in, view stock, enter the card number, item and quantity for a purchase, and review the shop's transaction log.
- Beneficiary: log in, view card details, monthly entitlement and history, and submit or track a grievance.

Geo Tag Photos with Guide

[paste geo-tagged photo with guide here]

Review Forms with Guide Signatures

[paste scanned review form with guide signature here]